

Universidad de Valladolid

Escuela de Ingeniería Informática

Grado en Ingeniería Informática

Lenguajes de Programación (LP)

# Memoria Práctica Bison y Lex

## Autoría

Este documento PDF tiene como objetivo el de documentar la entrega de la práctica número 2 de la asignatura "Lenguajes de Programación" correspondiente al tercer curso de Ingeniería Informática en la Universidad de Valladolid.

**Autores:**

Ángel Antolín Caramazana
Marcos Gómez Medina
Miembros del grupo de
laboratorio L2 (Jueves de 16-18)

Marzo 2026

# Índice

**1. Introducción** **2**

**2. Estrategia de resolución y tiempo empleado** **2**

**3. Alcance de nuestra práctica** **2**

**4. Explicación de fichero Lex** **3**
4.1. Primer Bloque . . . . . 3
4.2. Segundo Bloque . . . . . 3

**5. Explicación del archivo Bison** **6**
5.1. Primer Bloque . . . . . 6
5.2. Segundo Bloque . . . . . 6
5.3. Tercer Bloque . . . . . 8

**6. Conclusiones** **10**

# 1. Introducción

Para esta segunda práctica, el objetivo es el mismo que en la primera: el de crear un intérprete de Java en base a un fichero con algunas restricciones. Sin embargo, para la anterior práctica, solo se podía utilizar Flex; de esta manera, se pudo observar una gran limitación debido a este, pues, al fin y al cabo, existía muy poquito código de Flex y mucho de C.

Es por ello que, para esta nueva práctica, se permite utilizar Bison, y ahora se tiene preferencia al código en Bison sobre el de C siempre y cuando sea posible. Para poder crear esto, será necesario crear dos ficheros, uno en Flex y otro en Bison.

# 2. Estrategia de resolución y tiempo empleado

Con el objetivo de poder avanzar en esta nueva versión, el código en Flex se ha modificado bastante; ya no contiene casi código en C. Una vez hecho eso, se intenta traspasar la lógica general al archivo en Bison, priorizando las funcionalidades si son primitivas en Bison.

El tiempo estimado de realización ha sido en total de unas 20 horas. En cuanto a la repartición de trabajo, el archivo en Flex y parte del archivo Bison se realizó de manera conjunta entre ambos miembros del grupo; la finalización del segundo archivo la realizó Ángel Antolín y la memoria corre a cargo de Marcos Gómez. El trabajo realizado se hizo principalmente de manera presencial y cuando no era posible se utilizaba Discord como herramienta de comunicación.

# 3. Alcance de nuestra práctica

Para esta nueva versión, se mantienen las funcionalidades de la anterior práctica:

- **Aritméticas:** Soporta evaluación matemática completa con jerarquía de operadores y uso de paréntesis.
- **Booleanas:** Implementación de expresiones lógicas con jerarquía, uso de paréntesis y operadores relacionales.
- **Cadenas de texto (Strings):** Permite la concatenación de strings puros mediante el operador suma.
- **Incremento/Decremento:** Detecta operadores como `a++` o `--a`, pero exclusivamente cuando se encuentran aislados (no se soporta en asignaciones complejas como `b = a++`).
- **Operadores compuestos:** Soporte para operaciones como `+=`, permitiendo cualquier expresión matemática en la parte derecha de la igualdad.
- **Estructuras de control:** Funcionalidad para sentencias de if simple

# 4. Explicación de fichero Lex

## 4.1. Primer Bloque

En el primer bloque de Flex se han incluido las librerías `stdio.h`, `stdlib.h` y `string.h` para la manipulación de cadenas y memoria. También, se incluye `y.tab.h`, que permite a Flex reconocer los identificadores de los tokens definidos en Bison.

## 4.2. Segundo Bloque

En el archivo Lex se encuentran una serie de definiciones simples. Como se puede observar, la primera definición corresponde a los dígitos; luego, a través de los dígitos, podemos crear una nueva definición que consideramos como "Número", en la cual se contemplan tanto números positivos como negativos, así como valores decimales también.

```
%}

DIGITOS [0-9]
NUMERO [+-]?{DIGITOS}+(\.{DIGITOS}+)?[fFlL]?
CADENA \"[^\"]*\"
ID [a-zA-Z][a-zA-Z0-9_]*

%x COMENTARIO

%%
```

Figura 1: Definiciones en Flex (dígitos y números).

Si continuamos encontramos la cadena; es básicamente todo aquello que comience con comillas ("), continúe con algo que no sea una comilla y acabe con ella. Por otro lado, el id busca cualquier letra desde minúscula a mayúscula seguida de otras letras, números e inclusive el guion bajo (_).

```
%%

"/*"                {BEGIN(COMENTARIO);}
<COMENTARIO>"*/"    {BEGIN(INITIAL);}
<COMENTARIO>.|\n    {}

"//".*              {}
```

Figura 2: Regla para ignorar comentarios.

Finalmente, se establece el comentario como estado exclusivo. De esta manera se ignorarán los comentarios, ya sea de manera multilínea o de una sola, ya que cuando se detecta un comentario se activa el estado de COMENTARIO, del cual no se sale hasta que se detecta algo seguido de "*/", donde ahí ya regresa al "estado inicial".

En referencia a las reglas, se han realizado una serie de modificaciones en relación a la primera práctica:

- **Primeras reglas:** permiten ir construyendo algunas de las "piezas" más sencillas que no requieren de mucha más complejidad, como por ejemplo el *public class*, el *main*, acceder a un *if*, etc.

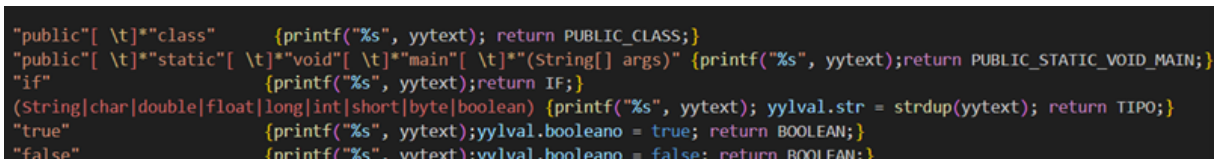



Figura 3: Reglas simples en Flex.

- **Reglas para numeros, cadenas e id:** Esta regla, lo primero que hace es imprimir el número; después asigna en la variable global (`yyval`) que es un número (tipo de dato 0, esto está en el archivo .y). Buscamos si contiene algo del tipo 9.3f o 9.3F, ya que de ese tipo será un nivel de FLOAT; lo mismo con long. Después, si aun así detectamos decimal en el texto, será double; en cualquier otro caso, entero.

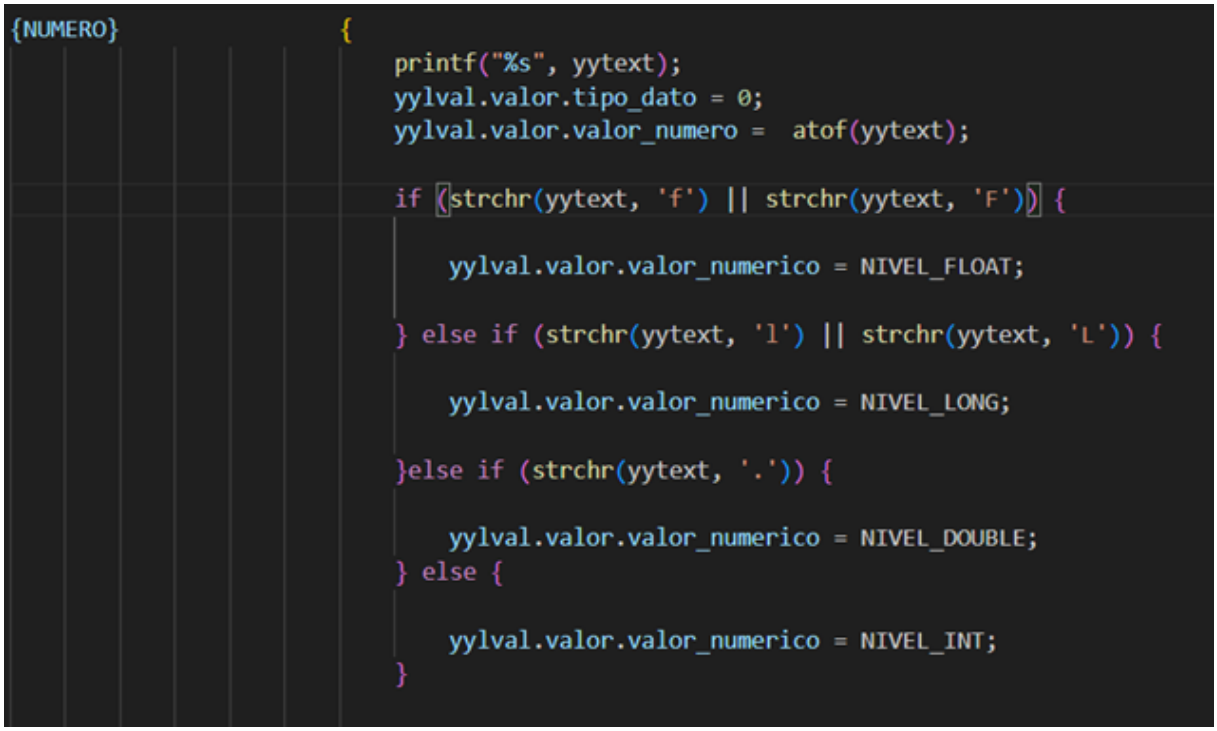



Figura 4: Regla Numeros Flex

Para cadenas seguimos los mismos pasos que anteriormente: imprimimos lo que hayamos leído; en caso de cadena asignamos el finalizador de cadena y duplicamos la salida, sin la comilla del principio y, como hemos puesto el carácter finalizador en

la posición donde se encontraba la comilla del final, no se pasa ninguna de las dos comillas. Para el identificador se sigue la misma lógica, solo que comprobando que no tenga una longitud superior a 200 caracteres.

```
{CADENA}        {
                    printf("%s", yytext);
                    yytext[strlen(yytext) - 1] = '\0';
                    yylval.str = strdup(yytext + 1);
                    return CADENA;
                }
{ID}            {
                    printf("%s", yytext);
                    if(strlen(yytext) > 200) {
                        yyerror("Identificador demasiado largo");

                    }
                    yylval.str = strdup(yytext);
                    return ID;
                }
```

Figura 5: Reglas Cadena e ID.

- **Reglas para Tokens:** Por último, estas reglas se encargan de devolver los tokens correspondientes a los simbolos posibles que pueden aparecer; en caso de tabulado o espacios se imprimen tal cual.

```
"+"             {printf("%s", yytext); return MAS;}
"-"             {printf("%s", yytext); return MENOS;}
"*"             {printf("%s", yytext); return POR;}
"/"             {printf("%s", yytext); return DIV;}
"%"             {printf("%s", yytext); return MOD;}
"=="            {printf("%s", yytext); return IGUAL_IGUAL;}
"="             {printf("%s", yytext); return IGUAL;}
"+="            {printf("%s", yytext); return MAS_IGUAL;}
"-="            {printf("%s", yytext); return MENOS_IGUAL;}
"*="            {printf("%s", yytext); return POR_IGUAL;}
"/="            {printf("%s", yytext); return DIV_IGUAL;}
"%="            {printf("%s", yytext); return MOD_IGUAL;}
"++"            {printf("%s", yytext); return MAS_MAS;}
"--"            {printf("%s", yytext); return MENOS_MENOS;}
"&&"            {printf("%s", yytext); return AND;}
"||"            {printf("%s", yytext); return OR;}
"!"             {printf("%s", yytext); return NOT;}
">"             {printf("%s", yytext); return MAYOR;}
"<"             {printf("%s", yytext); return MENOR;}
">="            {printf("%s", yytext); return MAYOR_IGUAL;}
"<="            {printf("%s", yytext); return MENOR_IGUAL;}
"!="            {printf("%s", yytext); return DIFERENTE;}
"("             {printf("%s", yytext); return PARENTESIS_IZQ;}
")"             {printf("%s", yytext); return PARENTESIS_DER;}
"{"             {printf("%s", yytext); return LLAVE_IZQ;}
"}"             {printf("%s", yytext); return LLAVE_DER;}
";"             {printf("%s", yytext); return PUNTO_Y_COMA;}

[ \t\n]+         {printf("%s", yytext);}
.                {printf("%s", yytext); return yytext[0];}
```

Figura 6: Reglas Tokens Flex.

Una vez tenemos esto, hemos conseguido generar los tokens correspondientes; a través de una variable global (`yyval`) se pasan los valores, y el token es aquello que retornamos.

# 5. Explicación del archivo Bison

## 5.1. Primer Bloque

En esta seccion tenemos un **code requires** necesario para que el archivo lex pueda hacer uso de esta seccion, donde se define los niveles numéricos para asegurar que se pueda operar sin casteos y la estructura que usaremos para guardar el tipo de las expresiones que se encuentra bison y así poder operar correctamente con ellas.

```
#define NIVEL_BYTE 1
#define NIVEL_SHORT 2
#define NIVEL_INT 3
#define NIVEL_LONG 4
#define NIVEL_FLOAT 5
#define NIVEL_DOUBLE 6

typedef struct {

    int tipo_dato; // 0: número, 1: cadena, 2: booleano
    int valor_numerico; // Para numeros, 1: byte, 2: short, 3: int, 4: long, 5: float, 6: double
    double valor_numero;
    char valor_cadena[512];
    bool valor_booleano;
}Valor;
```

Figura 7: Niveles de jerarquía y struct de datos.

Continuamos con los includes necesarios como son: `stdio.h`, `stdlib.h`, `string.h` y `stdbool.h`, además de declarar las variable globales externas `yyin`, `yylex`, `yyparse` e `yyerror`, provenientes de lex y necesarias en Bison.

Finalmente, en esta sección encontramos la declaración de nuestra tabla de símbolos que guardará las variables con su tipo y valor y la declaración de las funciones C que incluimos en el tercer bloque y que posteriormente serán explicadas.

```
typedef struct {
        char tipo[20];
        char id[201];
        char valor[512];
    } Variable;
```

Figura 8: Tabla de simbolos.

## 5.2. Segundo Bloque

Comienza con la declaracion del `union` que permitiráa Bison asignar los tipos correspondientes a los tokens, ya sea `char*`, `bool` o nuestro propio struct `Valor`.

Continuamos con la declaración de los tokens que recibe de lex y la precedencia de los operadores, ya sea de izquierda a derecha o la propia precedencia entre sumas y multiplicaciones por ejemplo.

```
%token <valor> NUMERO
%token <str> TIPO ID CADENA
%token <booleano> BOOLEAN

%token PUBLIC_CLASS PUBLIC_STATIC_VOID_MAIN IF
%token MAS MENOS POR DIV MOD
%token MAS_IGUAL MENOS_IGUAL POR_IGUAL DIV_IGUAL MOD_IGUAL
%token MAS_MAS MENOS_MENOS
%token IGUAL_IGUAL IGUAL DIFERENTE MAYOR MENOR MAYOR_IGUAL MENOR_IGUAL
%token AND OR NOT
%token PARENTESIS_IZQ PARENTESIS_DER LLAVE_IZQ LLAVE_DER PUNTO_Y_COMA

%type <valor> expresion expresion_logica

%left OR
%left AND
%left IGUAL_IGUAL DIFERENTE
%left MAYOR MENOR MAYOR_IGUAL MENOR_IGUAL
%left MAS MENOS
%left POR DIV MOD
%right NOT MAS_MAS MENOS_MENOS
```

Figura 9: Tokens y precedencia Bison.

Finalmente, una breve descripción de lo que hacen las reglas de este bloque:

- `programa`: Nuestro símbolo inicial de la gramática, define la estructura básica del programa Java con su declaración de clase, el main y el conjunto de sentencias contenidas en él.
- `Primera regla "sentencias"`: Se trata de una regla que es recursiva por la izquierda y que nos permite que los programas sean tanto una única intrucción como infinidad de ellas.
- `Segunda regla sentencia`: Define las diferentes operaciones individuales que puede interpretar y ejecutar el programa: una declaración, una asignación, un incremento/decremento o un condicional. Todas menos condicional terminan con un punto y coma (`;`) y llaman a la función `imprimir_variables()` para imprimir la tabla de símbolos actual.
- `declaracion`: Se encarga de la creación de nuevas variables. Evalúa la expresión a la derecha del igual (`=`) y comprueba que sea una declaración válida. En el caso de valores numéricos, comprueba mediante funciones auxiliares (como `cabe_en_byte` o `cabe_en_short`) que el dato no exceda la capacidad del tipo declarado antes de invocar a `guardar_variable()`.
- `asignacion`: Gestiona la modificación de variables ya existentes. Soporta la asignación simple y los operadores compuestos (`+=`, `-=`, `*=`, `/=`, `%=`). Comprueba que los tipos sean compatibles y al igual que la declaración para númericos comprueba si cabe en el lugar de guardado antes de llamar a `actualizar_variable()`.

- `incremento_decremento`: Aporta azúcar sintáctico para las operaciones de incremento (`++`) y decremento (`--`) aisladas. Verifica que el identificador corresponda a una variable numérica y actualiza su valor sumando o restando una unidad de forma directa.

- `expresion`: Se encarga de resolver la jerarquía y el cálculo de operaciones matemáticas (`+`, `-`, `*`, `/`, `%`) y concatenacion de Strings.

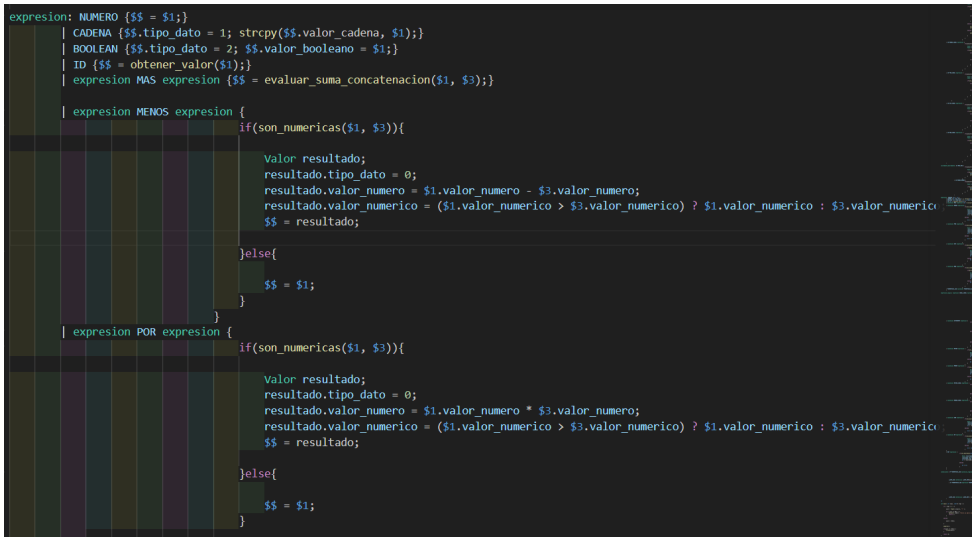



Figura 10: Ejemplo reglas expresion.

En la foto se aprecia la estructura de esta regla, para el resto de operadores la estructura es la misma, cambiando el cálculo y por ejemplo para la division se comprueba además que no se divida por 0.

- `expresion_logica`: Específica para evaluar comparaciones relacionales (`==`, `!=`, `>`, `<`, `>=`, `<=`) y operaciones booleanas (`AND`, `OR`, `NOT`). Garantiza que se estén comparando tipos compatibles y retorna siempre un valor estructurado de tipo booleano (`tipo_dato = 2`).

- `condicional`: Implementa la estructura de control de flujo `if`. Evalúa si la `expresion_logica` entre paréntesis es verdadera. En caso de ser falsa, desactiva temporalmente la variable global `ejecutar` (pasándola a `false`); esto permite que el intérprete siga analizando las sentencias del bloque interno sin llegar a ejecutar el guardado o actualizacion de las variables. Al cerrar las llaves del bloque, se restaura la variable a `true`.

## 5.3. Tercer Bloque

Aquí se implementan las funciones previamente declaradas en el primer bloque y que serán utilizadas por las reglas de Bison:

- `main`: Punto de entrada al programa y encargado de utilizar `yyin` para abrir el fichero Java de entrada.

- `yyerror`: Imprime los mensajes de error sintácticos que se produzcan a lo largo del programa.

- `imprimir_variables`: Recorre el array global de la tabla de símbolos `variables` y lo imprime. Muestra el tipo, el identificador y el valor formateado de cada variable registrada hasta el momento.
- `guardar_variable`: Se encarga de instanciar nuevas variables en la tabla de símbolos. Recibe el tipo, el identificador y una estructura `Valor`. Dependiendo del campo `tipo_dato` (0 para números, 1 para cadenas, 2 para booleanos), formatea el dato y lo guarda en el array `variables`, incrementando posteriormente el contador `num_variables`.
- `actualizar_variable`: Modifica el valor de una variable previamente declarada. Busca el identificador en la tabla de símbolos y, si lo encuentra, sobrescribe su contenido aplicando el formato correspondiente según el tipo de dato que se le asigne, de forma análoga a `guardar_variable`.
- `obtener_valor`: Dado un identificador, recorre la tabla de símbolos buscándolo. Si lo encuentra, determina su tipo mediante comprobaciones de cadena (`strcmp`) y devuelve una estructura `Valor`. Si no lo encuentra, lanza un error notificando que la variable no está declarada.
- `es_numerica` **y** `son_numericas`: Comprueban que el `tipo_dato` de uno o dos `Valor`es sea igual a 0. En caso negativo, lanzan un error a través de `yyerror`. Utilizado por las reglas que evaluan las expresiones aritméticas.
- `son_booleanas`: Similar a las anteriores, pero verifica que ambos valores tengan `tipo_dato == 2`, garantizando que operaciones como `AND` u `OR` solo se apliquen sobre tipos lógicos.
- `division_por_cero`: Se invoca antes de realizar divisiones (`/`) o módulos (`%`) para evitar fallos críticos en la ejecución del intérprete.
- `cabe_en_byte` **y** `cabe_en_short`: Verifican si un valor de tipo entero (nivel INT) puede almacenarse de forma segura dentro de los límites físicos de un tipo `byte` (-128 a 127) o `short` (-32768 a 32767) sin pérdida de precisión.
- `tipo_numerico`: Asigna un valor jerárquico a los diferentes tipos numéricos primitivos. Al recibir el nombre del tipo como cadena (ej. ïnt", "double"), devuelve una constante (como `NIVEL_INT`) que permite a otras funciones saber el rango y la precedencia del tipo.
- `evaluar_suma_concatenacion`: Sobrecarga el operador de suma (`+`). Primero comprueba si alguno de los operandos es de tipo cadena (`tipo_dato == 1`); si es así, convierte el otro operando (sea número o booleano) a cadena y los concatena usando `snprintf`. Si ambos son numéricos, realiza una suma matemática estándar y promociona el tipo del resultado al mayor de los dos niveles implicados.

Por finalizar la sección de Bison, añadimos que el manejo de errores como puede ser tratar de dividir por 0, guardar un valor string en una variable numérica, intentar guardar un valor double en una variable short, etc. Lo hemos hecho de forma de que el programa notifica de dicho error a través de yyerror pero no detiene la ejecución del programa, evitando que se modifiquen las variables afectadas por el error pero permitiendo que el programa se ejecute hasta el final sacando todos los errores que encuentra, imitando el funcionamiento de un compilador.

# 6. Conclusiones

Tras haber hecho la práctica hemos comprendido mucho más las limitaciones de Flex y lo bien que se combina con Bison para realizar el trabajo.

La mayor de las mejoras frente a la anterior práctica fue la posibilidad de seguir la jerarquía de operaciones directamente en Bison sin necesidad de utilizar funciones complejas en C, como nos ocurrió en la primera práctica.

Entre las dificultades encontradas podríamos destacar:

- Comunicación entre lex y bison para utilizar las variables y el struct, pues en un principio no sabíamos porque no funcionaba (solucionado con el code requires)
- Utilización del struct Valor para el manejo de tipos dentro de Bison, pues no conseguíamos evitar los errores que se producían al tratar de hacer operaciones con varios tipos incluidos.